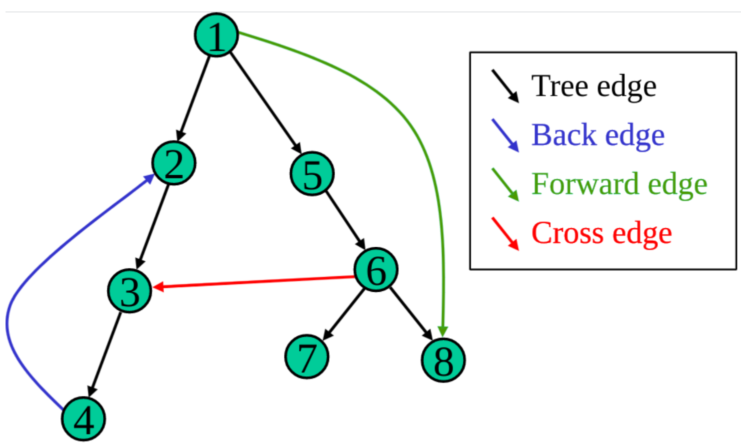
CS 253 DFS tree edge

Problem 3 (DFS Edges, 20 points). See the program DepthFirstEdge.java Download DepthFirstEdge.javaLinks to an external site.. Suppose we are given a digraph G with no parallel edges. When we construct "dfe = new DepthFirstEdges(G)", the constructor performs a DFS traversal of G. The dfe object then provides four methods, which are meant to classify the edges of G. For each edge (v,w) of G, exactly one of the following should be true:

- **dfe.isTreeEdge(v,w):** true if v is the parent of w in the tree. Equivalently, v discovered w on this edge.
- **dfe.isForwardEdge(v,w):** true if v is an ancestor of w in the tree, but (v,w) is not a tree edge.
- **dfe.isBackEdge(v,w):** true if w is an ancestor of v, including possibly v==w. This implies a cycle.
- **dfe.isCrossEdge(v,w):** all other edges.

This example image is from wikipedia (Links to an external site.) (the vertices are numbered in discovery order):



During the DFS traversal, it computes t1[v], t2[v], and edgeTo[v] for every vertex v. Note that t1[v] and t2[v] act like "timestamps", indicating when each dfs(G,v) started and finished. That is enough information to write the following expressions.

As given, the four methods each return "false", which is incorrect. You want to replace each "false" with a correct boolean expression, which executes in O(1) time.

(a) give a correct boolean expression for isTreeEdge(v,w).

(b) give a correct boolean expression for isForwardEdge(v,w).

(c) give a correct boolean expression for isBackEdge(v,w).

(d) give a correct boolean expression for isCrossEdge(v,w). Try to keep it simple!

(e) Argue that there are no two vertices x and y such that t1[x] < t1[y] < t2[x] < t2[y].   (There may or may not be an edge between x and y, it does not matter.)

Expected answer format: We expect simple boolean Java expression for parts (a) through (d). These can be short with at most a line. In particular, the expression for (d) should be simpler than saying "the negation of the other three cases". They may be handwritten, but I recommend testing them. They should be independent; that is, no one of the four methods should call another. For part (e) we expect a short argument, maybe two sentences.